

Enterprise-Grade AI-Native Collaborative Resume Platform: Master PRD & Architecture

This document serves as the comprehensive technical specification and implementation framework for the AI-native collaborative resume platform within the Graviton ecosystem. The system focuses on transparency, cinematic real-time interactions, and enterprise-grade collaborative editing.¹

1. Product Vision and UX Philosophy

The platform is designed to transition the resume from a static document into a dynamic professional identity ecosystem.

- **Human-AI Collaboration:** Users work *with* AI rather than being replaced by it. Every AI modification is visible, reversible, and controllable.¹
- **Cinematic Experience:** Real-time AI interactions should feel premium, calm, and intelligent, utilizing streaming updates and visual diffs.¹
- **Transparency:** All AI-generated content must be grounded in the candidate's actual history to prevent "hallucinated" credentials.

2. Core Feature Specifications

2.1. Real-time Collaborative AI Editor

- **Inline Autocomplete (Ghost-text):** Implementation of ghost-text completions similar to GitHub Copilot, providing predictive text suggestions as the user types.
- **Visual AI Diff Rendering:** Highlighting changes suggested by the AI in a "track changes" format, allowing users to accept or reject specific modifications.¹
- **WebSocket Streaming:** A persistent socket connection to handle streaming AI modifications, patch-based updates, and real-time draft synchronization.

2.2. Recruiter Intelligence & ATS Optimization

- **ATS Scoring:** Real-time analysis of the resume against 23+ ATS checkpoints to ensure compatibility with modern hiring engines.
- **Attention Heatmap Simulation:** A visual overlay based on eye-tracking research (F-pattern) that predicts where recruiters will focus their attention in the first 6-7 seconds.
- **Keyword Optimization:** Intelligent extraction of missing competencies from job descriptions to naturally weave them into experience bullets.

2.3. Enterprise Systems

- **Organization Template System:** Support for branded templates with locked sections, custom fonts, and organization-specific formatting constraints.¹
- **Draft Recovery:** Utilizing IndexedDB for local persistence and cloud synchronization to ensure offline editing support and snapshot versioning.

3. Technology Stack Taxonomy

The project utilizes a modern, performance-oriented stack designed for real-time state management and agentic orchestration.

Layer	Recommended Technology	Role in System
Frontend Framework	Next.js (App Router), TypeScript	Core application structure and SSR capabilities.
Rich Text Editor	Lexical Editor	Handles ghost-text, collaborative nodes, and structured state.
State Management	Zustand	Lightweight, hook-based global state for the editor and UI.
Animations	Framer Motion	Provides "cinematic" UI transitions and streaming effects. ¹
Backend API	Python, FastAPI	High-performance async API for LLM orchestration.
Orchestration	LangGraph, LangChain	Managing multi-step AI workflows and stateful graphs.
Task Queue	Celery, Redis	Handles long-running AI background tasks and

		caching.
Database	PostgreSQL (pgvector)	Relational data and vector embeddings for semantic search.

4. Architectural Design

4.1. The Multi-Agent Workforce (LangGraph)

The "brain" of the platform is a stateful graph where specialized nodes handle different phases of the resume lifecycle:

- Extraction Node:** Parses raw input into a structured professional schema (JSON).
- Analysis Node:** Performs a gap analysis against target job descriptions.
- Optimization Node:** Refines bullet points using the STAR (Situation-Task-Action-Result) method.
- Validation Node:** Checks for PII (Personally Identifiable Information) and ensures no "fake" stories were invented.

4.2. Real-time Synchronization Flow

To support collaborative editing, the architecture follows a CRDT (Conflict-free Replicated Data Type) pattern using **Yjs** or **Liveblocks** integrated with the Lexical editor.

- Client Side:** Lexical maintains an immutable EditorState. Changes trigger a diff that is sent over WebSockets.
- Server Side:** FastAPI receives patches, updates the PostgreSQL/Redis source of truth, and broadcasts the update to other connected clients.

5. API Catalog (Standard Endpoints)

The backend exposes the following primary endpoints for enterprise integration:

Endpoint	Method	Purpose
/api/v1/resumes	POST	Create a new resume instance.
/api/v1/resumes/{id}/rewrite	POST	Trigger an AI-driven rewrite of

		a specific section.
/api/v1/ats/analyze	POST	Return ATS score and keyword gaps.
/api/v1/templates/upload	POST	Organization-level branded template ingestion.
/ws/resume/{id}	WS	Real-time collaborative socket connection.

6. Implementation Roadmap

- **Phase 1: Foundation (MVP):** Setup FastAPI/Next.js skeleton, implement basic Lexical editor, and build the resume extraction RAG pipeline.
- **Phase 2: Real-time & Collab:** Integrate WebSockets for streaming AI suggestions and implement basic ghost-text logic.
- **Phase 3: Intelligence Layer:** Deploy the attention heatmap simulation and multi-agent job-fit assessment workflows.
- **Phase 4: Enterprise Scaling:** Build the organization template system and implement advanced observability via LangSmith.

7. Conclusion

The differentiator of this platform is not just AI generation, but the **real-time collaborative experience** combined with recruiter intelligence and cinematic visuals. By utilizing a stack centered on Next.js, Lexical, and LangGraph, the platform provides an intelligent, transparent workspace that empowers the candidate while satisfying enterprise-grade requirements.¹

Works cited

1. Enterprise_Grade_AI_Resume_Platform_PRD_RFC.pdf